Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ФУНДАМЕНТАЛЬНЫЕ НАУКИ»

КАФЕДРА «МАТЕМАТИЧЕСКОЕ МОДЕЛИРОВАНИЕ»

Лабораторная работа №9
по дисциплине «Типы и структуры данных»

Тема Конечные автоматы

Студент Михайлов А.К.

Группа ФН12-31Б

Преподаватели Строганов Ю.В., Волкова Л.Л.

Москва, 2025

Содержание

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Конечные автоматы 5

1.2 Формат WebAssembly 5

1.3 LEB-128 6

2 Конструкторская часть **7**

2.1 Класс WasmParser 7

2.2 Декодеры инструкций по группам 7

2.3 Структура `ModeSet` 8

2.4 Взаимодействие компонентов 8

2.5 Автомат, описывающий разбор файла 9

3 Технологическая часть **11**

3.1 Выбор средств реализации 11

3.2 Реализация алгоритмов 11

3.3 Тестирование 12

ЗАКЛЮЧЕНИЕ **18**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **19**

ПРИЛОЖЕНИЕ А **20**

ВВЕДЕНИЕ

В рамках данной лабораторной работы получено следующее задание:

Выполнить разбор секции формата $WebAssembly$ с $id = 10$ с использованием конечного автомата. Реализовать в программе поддержку Control Instructions, Table Instructions, Memory Instructions, Numeric Instructions [3].

Целью работы является получение навыков работы с парсингом бинарных форматов и структурным анализом инструкций WebAssembly. Поставлены следующие задачи:

1) привести всю необходимую теорию по конечным автоматам, формату $WebAssembly$ и кодированию LEB-128;
2) разработать алгоритм парсинга секции кода с $id = 10$ с поддержкой выборочного декодирования групп инструкций, приведенных в задании;
3) выполнить реализацию данного алгоритма на языке $C++$;
4) провести тестирование реализации на различных режимах работы программы.

1 Аналитическая часть

1.1 Конечные автоматы

Абстрактный дискретный исполнитель, преобразующий последовательность входных сигналов в последовательность выходных сигналов так, что значение символа на выходе исполнителя зависит не только от значения входного символа, но и от предыдущей последовательности входных символов (предыстории), называют автоматом.

Любые две предыстории эквивалентны, если для каждой их них любая последовательность входных символов будет приводить к одинаковой последовательности выходных символов. Если количество эквивалентных предысторий конечно, то соответствующий им автомат называют конечным автоматом [1].

Конечный автомат может быть задан как пятерка:

$$M = (Q, E, \phi, q_0, F),$$

где Q — множество состояний автомата; E — множество дуг; ϕ — функция разметки (весовая функция), причем для каждой дуги $e \in E$ ее метка $\phi(e) \subseteq V$, при этом подмножество $\phi(e)$ не пусто; $q_0 \in Q$ — начальное состояние; $F \subseteq Q$ — подмножество заключительных состояний [2].

Процесс преобразования входных сигналов в выходные конечным автоматом состоит из следующих действий:

1) поступление входного сигнала;
2) переход автомата из одного состояния в другое;
3) выдача входного сигнала.

1.2 Формат WebAssembly

WebAssembly — это «безопасный, переносимый, низкоуровневый формат кода, спроектированный с целью эффективного исполнения и компактного представления» [3]. Основной целью разработки такого формата является появление возможности создания высокопроизводительных приложений для Веба, при этом формат разрабатывается как пригодный для использования и в других средах.

Модули WebAssembly — это базовые единицы для развертывания и выполнения. Они состоят из определений типов, функций, таблиц, памяти и глобальных данных, а также могут импортировать и экспортировать внешние объекты.

Двоичная структура модуля организована в виде секций, каждая из которых соответствует определённому компоненту (например, типы, функции, код, данные). Секции идентифицируются числовым ID, имеют размер и содержимое, специфичное для каждой из них. Они могут отсутствовать, что равнозначно пустой секции.

Особое место занимают пользовательские секции (ID 0), которые могут содержать от-

ладочную информацию или сторонние расширения. Они могут располагаться в любом месте модуля и игнорируются при выполнении.

Перед всеми секциями идёт заголовок из 8 байт: магическое число 0x00 0x61 0x73 0x6d и версия формата 0x01 0x00 0x00 0x00.

Выполнение кода строится на стековой машине: инструкции последовательно обрабатывают данные в стеке. Поддерживаются типы: целые и вещественные числа разных размеров, векторы, ссылки на функции и объекты внешней среды. Управление потоком осуществляется через структурированные блоки, циклы и условия.

1.3 LEB-128

Целые числа, используемые в файлах формата WebAssembly кодируются в формате LEB-128. Для преобразование целого числа к формату LEB-128 необходимо его двоичное представление разбить на группы по 7 бит, начиная с младшего разряда, дополняя незначащими нулями при необходимости. Полученные группы бит записываются каждая в свой байт, начиная с группы, содержащей младшие разряды. К каждой группе слева приписывается бит, являющийся признаком окончания числа: 1, если число не заканчивается в этом байте, и 0 иначе [4].

Вывод

В данном разделе описана вся используемая в данной лабораторной работе теория.

2 Конструкторская часть

2.1 Класс WasmParser

`WasmParser` — основной класс для парсинга WebAssembly-файлов с поддержкой выбора режима декодирования инструкций.

Внутренняя структура

1) `wasm_data` — бинарные данные WASM-файла;
2) `current_offset` — текущая позиция чтения в данных;
3) `mode` — структура `ModeSet`, определяющая, какие группы инструкций должны быть декодированы.

Приватные вспомогательные методы

1) `read_leb_u32()` — чтение LEB128-числа (беззнаковое 32-битное);
2) `read_leb_u64()` — чтение LEB128-числа (беззнаковое 64-битное);
3) `read_leb_s32()` — чтение LEB128-числа (знаковое 32-битное);
4) `read_u8()` — чтение одного байта;
5) `read_f32_ieee()` — чтение числа `f32` в формате IEEE-754;
6) `read_f64_ieee()` — чтение числа `f64` в формате IEEE-754;
7) `blocktype_to_string()` — определение строкового представления типа блока.

Публичные методы парсера

1) `verify_wasm_header()` — проверка магического числа и версии WASM-файла;
2) `find_section_payload()` — поиск секции по идентификатору;
3) `parse_code_section_10()` — парсинг секции кода (ID=10) с учётом выбранного режима.

2.2 Декодеры инструкций по группам

В зависимости от режима работы, программа использует одну или несколько следующих функций декодирования:

Декодер контрольных инструкций (`decode_control`)

Обрабатывает инструкции управления: `block`, `loop`, `if`, `br`, `br_if`, `br_table`, `return`, `call`, `call_indirect`, `throw`, `try_table`, `br_on_null`, `br_on_non_null`, а также расширенные инструкции `br_on_cast` и `br_on_cast_fail` (подопкоды 24 и 25 для префикса `0xFB`).

Декодер табличных инструкций (`decode_table`)

Поддерживает инструкции работы с таблицами:

1) `table.get` (0x25);

2) `table.set` (0x26).

Декодер числовых инструкций (`decode_numeric_part12`)

Обрабатывает:

1) константы `i32.const`, `i64.const`, `f32.const`, `f64.const`;

2) операции сравнения и проверки для типов `i32`, `i64`, `f32`, `f64`.

Декодер инструкций памяти (`decode_memory`)

Поддерживает:

1) загрузки `i32.load`, `i64.load`, `f32.load`, `f64.load`, а также загрузки с преобразованием размера;

2) сохранения `i32.store`, `i64.store`, `f32.store`, `f64.store`, а также сохранения с усечением;

3) управление памятью `memory.size`, `memory.grow`.

2.3 Структура `ModeSet`

Структура `ModeSet` определяет, какие группы инструкций должны быть обработаны. Поддерживаются следующие режимы:

1) `mode=12` — только контрольные инструкции;

2) `mode=13` — табличные и числовые инструкции;

3) `mode=14` — инструкции работы с памятью;

4) `mode=all` — все перечисленные группы.

Вспомогательные функции

1) `parse_mode()` — парсинг аргументов командной строки для определения режима работы;

2) `valtype_name()` — возвращает строковое имя типа значения по его байтовому коду;

3) `print_indent()` — вывод отступов для форматированного вывода вложенных блоков.

2.4 Взаимодействие компонентов

1) программа читает бинарный WASM-файл и проверяет заголовок;

2) определяется режим работы через аргумент `-mode`;

3) находится секция кода (ID=10);
4) читаются локальные переменные;
5) инструкции декодируются в соответствии с выбранным режимом;
6) результат выводится в консоль с отступами, отражающими структуру блоков.

2.5 Автомат, описывающий разбор файла

На рисунке 2.1 представлено графическое предстваление автомата, описывающего разбор файла согласно варианту.

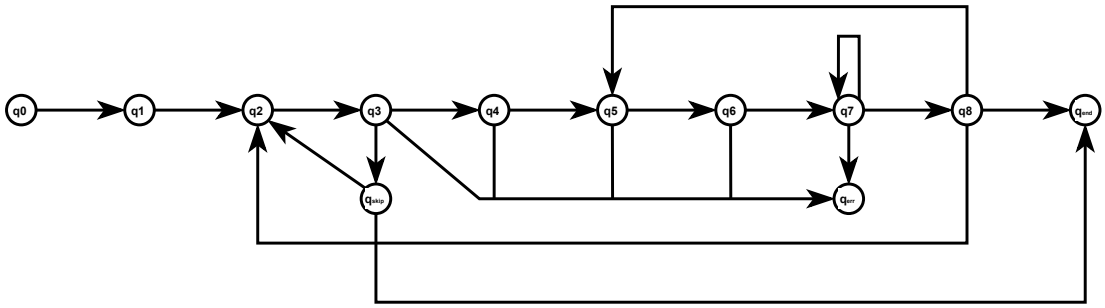

Рисунок 2.1 — Рассматриваемый автомат

Состояния автомата:

1) q_0 — начальное состояние, открытие фойла и установка указателя чтения в начало;
2) q_1 — чтение заголовка WASM, обработка первых 8 байт;
3) q_2 — чтение ID секции;
4) q_3 — чтение размера секции;
5) q_{skip} — пропуск секции, указатель чтения смещается на section_size байт вперед;
6) q_4 — чтение количества функций в Code Section;
7) q_5 — чтение размера тела функции;
8) q_6 — чтение локальных переменных;
9) q_7 — разбор инструкций функции;
10) q_8 — завершение функции, проверка, равно ли число прочитанных байт числу body_size;
11) q_{end} — завершающее состояние;
12) q_{err} — состояние ошибки.

Условия перехода состояний автомата:

1) q_0 — если файл успешно открыт, переход в q_1, переход в q_{err}, иначе;
2) q_1 — переход в q_2, если сигнатура корректна, переход в q_{err}, иначе;
3) q_2 — переход в q_{err}, если возникла ощибка чтения, переход в q_3, иначе;
4) q_3 — переход в q_4, если section_id = 10, переход в q_{err}, если возникла ошибка, переход в q_{skip}, иначе;

5) q_{skip} — переход в q_2, если файл не закончился, переход в q_{end}, иначе;

6) q_4 — переход в q_{err}, если возникла ошибка, переход в q_5, иначе;

7) q_5 — переход в q_{err}, если возникла ошибка, переход в q_6, иначе;

8) q_6 — переход в q_{err}, если возникла ошибка, переход в q_7, иначе;

9) q_7 — переход в q_7, если opcode $\neq$ end, переход в q_8, если opcode = end, переход в q_{err}, есди байты вышли за предел body_size;

10) q_8 — переход в q_5, если есть еще функции, переход в q_2, иначе.

Вывод

В данном разделе описана архитектура программы для парсинга секции кода WebAssembly с поддержкой выбора групп инструкций. Реализованы:

1) парсер WASM-файлов с навигацией по секциям;
2) декодеры для четырёх групп инструкций (control, table, numeric, memory);
3) гибкая система выбора режима работы через аргументы командной строки;
4) форматированный вывод с учётом вложенности блоков.

3 Технологическая часть

3.1 Выбор средств реализации

Для программной реализации использовалась среда разработки $VisualStudio$, язык программирования $C++$ и компилятор GCC из набора $MSYS2$.

3.2 Реализация алгоритмов

В листинге [A.1] приложении А представлена программная реализация описанных алгоритмов.

Программа реализована как консольное приложение, принимающее аргументы командной строки. Основной функционал включает:

1) чтение бинарного $WASM$-файла в память;
2) проверку заголовка $WASM$ (магическое число и версия);
3) поиск секции кода ($ID = 10$);
4) декодирование инструкций в соответствии с выбранным режимом;
5) форматированный вывод структуры функций с учётом вложенности блоков.

Режимы работы

Программа поддерживает четыре режима работы через аргумент `-mode`:

1) `12` — декодирование только контрольных инструкций;
2) `13` — декодирование табличных и числовых инструкций;
3) `14` — декодирование инструкций работы с памятью;
4) `all` — декодирование всех поддерживаемых групп инструкций.

Структуры данных

1) `ModeSet` — флаги для выбора групп инструкций.
2) `BlockStackEntry` — запись в стеке блоков для отслеживания вложенности.

Основные функции

1) `read_leb_u32()`, `read_leb_u64()`, `read_leb_s32()` — чтение $LEB128$-чисел;
2) `decode_control()` — декодирование контрольных инструкций;
3) `decode_table()` — декодирование табличных инструкций;
4) `decode_numeric_part12()` — декодирование числовых инструкций;
5) `decode_memory()` — декодирование инструкций памяти.;
6) `parse_code_section_10()` — основной парсинг секции кода.

3.3 Тестирование

В данном разделе приводится описание тестирования разработанной программы разбора бинарных модулей $WebAssembly$. Для проверки корректности работы используеюся специально сформированные тестовые файлы, структура которого подробно анализируется на побайтовом уровне.

Тест 1

Первый тест проводился на файле `mini_test.wasm`, содержащем две функции с инструкциями разных типов. Полный HEX-дамп файла представлен в листинге 3.1, а его побайтовая структура — в листинге 3.2. В листинге 3.3 приведен результат работы программы с данным файлом.

```
00 61 73 6d 01 00 00 00 01 04 01 60 00 00 03 02 01 00 0a 05
01 03 00 01 0b 00 0d 01 78 00 01 02 03 04 05 06 07 08 09 0a
```

Листинг 3.1 — Полный HEX-дамп файла mini_test.wasm

```
Размер файла: 40 байт (0x28)

=== ЗАГОЛОВОК WASM (8 байт) ===
Смещение 0x00: 00 61 73 6d  магическое число "\0asm"
Смещение 0x04: 01 00 00 00  версия формата = 1 (little-endian)

=== СЕКЦИЯ TYPE (ID=1, 4 байта payload) ===
Смещение 0x08: 01  ID секции Type
Смещение 0x09: 04  длина секции (payload = 4 байта)
Смещение 0x0A: 01  количество типов функций (1)
Смещение 0x0B: 60 00 00  тип #0: func () -> ()

=== СЕКЦИЯ FUNCTION (ID=3, 2 байта payload) ===
Смещение 0x0E: 03  ID секции Function
Смещение 0x0F: 02  длина секции (payload = 2 байта)
Смещение 0x10: 01  количество функций (1)
Смещение 0x11: 00  тип функции #0 (индекс 0 в секции Type)

=== СЕКЦИЯ CODE (ID=10, 5 байт payload) ===
Смещение 0x12: 0a  ID секции Code
Смещение 0x13: 05  длина секции (payload = 5 байт)
Смещение 0x14: 01  количество функций в секции кода (1)

=== ТЕЛО ФУНКЦИИ #0 (3 байта) ===
```

```
Смещение 0x15: 03  размер тела функции (3 байта)
Смещение 0x16: 00  количество деклараций локальных переменных (0)

=== БАЙТКОД ФУНКЦИИ #0 (2 байта) ===
Смещение 0x17: 01  инструкция nop
Смещение 0x18: 0b  инструкция end (конец функции)

=== СЕКЦИЯ CUSTOM (ID=0, 13 байт payload) ===
Смещение 0x19: 00  ID секции Custom
Смещение 0x1A: 0d  длина секции (payload = 13 байт)

Смещение 0x1B: 01        длина имени секции (1)
Смещение 0x1C: 78        имя секции: 'x'
Смещение 0x1D: 00 01 02 03 04 05 06 07 08 09 0a  данные секции (11 ба
 йт)
```

Листинг 3.2 — Побайтовая структура файла mini_test.wasm

```
CodeSection(id=10): functions=1

func[0]: body_size=3
locals_decls=0
nop
end
```

Листинг 3.3 — Результат работы программы

Тест 2

Второй тест проводился на файле `test_12_13_14.wasm`, содержащем две функции с инструкциями разных типов. Полный HEX-дамп файла представлен в листинге 3.4, а его побайтовая структура — в листинге 3.5. В листинге 3.6 приведен результат работы программы с данным файлом.

```
00 61 73 6d 01 00 00 00 01 09 02 60 00 01 7f 60 00 01 70
03 03 02 00 01 04 04 01 70 00 01 05 03 01 00 01 07 18 04
01 66 00 00 04 74 67 65 74 00 01 03 6d 65 6d 02 00 03 74
61 62 01 00 0a 22 02 19 00 41 00 41 7b 36 02 00 41 00 28
02 00 41 64 4a 04 7f 41 01 05 41 00 0b 0b 06 00 41 00 25
00 0b
```

Листинг 3.4 — Полный HEX-дамп файла test_12_13_14.wasm

```
Размер файла: 97 байт (0x61)

=== ЗАГОЛОВОК WASM (8 байт) ===
Смещение 0x00: 00 61 73 6d  магическое число "\0asm"
Смещение 0x04: 01 00 00 00  версия формата = 1 (little-endian)

=== СЕКЦИЯ TYPE (ID = 1, 9 байт payload) ===
Смещение 0x08: 01  ID секции Type
Смещение 0x09: 09  длина секции (9 байт)
Смещение 0x0A: 02  количество типов функций (2)

Тип #0: (func () -> i32)
Смещение 0x0B: 60  маркер func
Смещение 0x0C: 00  количество параметров = 0
Смещение 0x0D: 01  количество результатов = 1
Смещение 0x0E: 7f  тип результата i32

Тип #1: (func () -> funcref)
Смещение 0x0F: 60  маркер func
Смещение 0x10: 00  количество параметров = 0
Смещение 0x11: 01  количество результатов = 1
Смещение 0x12: 70  тип результата funcref

=== СЕКЦИЯ FUNCTION (ID = 3, 3 байта payload) ===
Смещение 0x13: 03  ID секции Function
Смещение 0x14: 03  длина секции (3 байта)
Смещение 0x15: 02  количество функций (2)
Смещение 0x16: 00  тип функции #0 (тип #0 из секции Type)
Смещение 0x17: 01  тип функции #1 (тип #1 из секции Type)

=== СЕКЦИЯ TABLE (ID = 4, 4 байта payload) ===
Смещение 0x18: 04  ID секции Table
Смещение 0x19: 04  длина секции (4 байта)
Смещение 0x1A: 01  количество таблиц (1)
Смещение 0x1B: 70  тип элементов таблицы funcref
Смещение 0x1C: 00  flags = 0 (только min)
Смещение 0x1D: 01  min = 1 элемент

=== СЕКЦИЯ MEMORY (ID = 5, 3 байта payload) ===
Смещение 0x1E: 05  ID секции Memory
```

```
Смещение 0x1F: 03  длина секции (3 байта)
Смещение 0x20: 01  количество областей памяти (1)
Смещение 0x21: 00  flags = 0
Смещение 0x22: 01  min = 1 страница (64 KiB)

=== СЕКЦИЯ EXPORT (ID = 7, 24 байта payload) ===
Смещение 0x23: 07  ID секции Export
Смещение 0x24: 18  длина секции (24 байта)
Смещение 0x25: 04  количество экспортов (4)

Экспорт #0: функция "f"
Смещение 0x26: 01        длина имени
Смещение 0x27: 66        'f'
Смещение 0x28: 00        kind = function
Смещение 0x29: 00        index = 0

Экспорт #1: функция "tget"
Смещение 0x2A: 04        длина имени
Смещение 0x2B: 74 67 65 74  "tget"
Смещение 0x2F: 00        kind = function
Смещение 0x30: 01        index = 1

Экспорт #2: память "mem"
Смещение 0x31: 03        длина имени
Смещение 0x32: 6d 65 6d  "mem"
Смещение 0x35: 02        kind = memory
Смещение 0x36: 00        index = 0

Экспорт #3: таблица "tab"
Смещение 0x37: 03        длина имени
Смещение 0x38: 74 61 62  "tab"
Смещение 0x3B: 01        kind = table
Смещение 0x3C: 00        index = 0

=== СЕКЦИЯ CODE (ID = 10, 34 байта payload) ===
Смещение 0x3D: 0a  ID секции Code
Смещение 0x3E: 22  длина секции (34 байта)
Смещение 0x3F: 02  количество функций (2)

=== ТЕЛО ФУНКЦИИ #0 ===
Смещение 0x40: 19  размер тела функции (25 байт)
```

```
Смещение 0x41: 00  количество локальных деклараций (0)

Байткод функции #0:
Смещение 0x42: 41 00        i32.const 0
Смещение 0x44: 41 7b        i32.const 123
Смещение 0x46: 36 02 00     i32.store align=2 offset=0
Смещение 0x49: 41 00        i32.const 0
Смещение 0x4B: 28 02 00     i32.load align=2 offset=0
Смещение 0x4E: 41 64        i32.const 100
Смещение 0x50: 4a           i32.gt_s
Смещение 0x51: 04 7f        if (result i32)
Смещение 0x53: 41 01        i32.const 1
Смещение 0x55: 05           else
Смещение 0x56: 41 00        i32.const 0
Смещение 0x58: 0b           end (if)
Смещение 0x59: 0b           end (функции)

=== ТЕЛО ФУНКЦИИ #1 ===
Смещение 0x5A: 06  размер тела функции (6 байт)
Смещение 0x5B: 00  количество локальных деклараций (0)

Байткод функции #1:
Смещение 0x5C: 41 00     i32.const 0
Смещение 0x5E: 25 00     table.get 0
Смещение 0x60: 0b        end
```

Листинг 3.5 — Побайтовая структура файла test_12_13_14.wasm

```
C:\labs\laba9\x64\Debug>Лаба_9.exe
C:\labs\test_12_13_14.wasm --mode all
CodeSection(id=10): functions=2

func[0]: body_size=25
locals_decls=0
i32.const 0
i32.const 123
i32.store align=2 offset=0
i32.const 0
i32.load align=2 offset=0
i32.const 100
i32.gt_s
if i32
i32.const 1
```

```
else
i32.const 0
end
end

func[1]: body_size=6
locals_decls=0
i32.const 0
table.get 0
end
```

Листинг 3.6 — Резуьтат работы программы

Анализ результатов

Функция #0:

1) содержит инструкции памяти (`i32.store`, `i32.load`);
2) содержит числовые инструкции (`i32.const`, `i32.gt_s`);
3) содержит контрольные инструкции (`if`, `else`, `end`);
4) правильно отображается вложенная структура блока `if`.

Функция #1:

1) содержит табличную инструкцию `table.get`;
2) корректно декодируется и выводится.

Вывод

В ходе тестирования программа продемонстрировала корректную работу во всех режимах:

1) корректно читает и проверяет $WASM$-файлы;
2) находит секцию кода ($ID = 10$);
3) декодирует инструкции в соответствии с выбранным режимом;
4) правильно отображает вложенную структуру блоков с отступами;
5) изолирует группы инструкций в соответствии с режимом (только выбранные типы инструкций выводятся);
6) обрабатывает все заявленные категории инструкций: контрольные, табличные, числовые и инструкции памяти.

Программа выполняет поставленную задачу: парсинг секции кода $WebAssembly$ с возможностью выбора групп инструкций для анализа. Все тесты пройдены успешно.

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы была достигнута цель работы, а именно получены навыки работы с парсингом бинарных форматов и структурным анализом инструкций $WebAssembly$. Также были выполнены все задачи:

1) приведена вся необходимая теория по конечным автоматам, формату $WebAssembly$ и кодированию LEB-128;
2) разработан алгоритм парсинга секции кода с $id = 10$ с поддержкой выборочного декодирования групп инструкций;
3) выполнена реализация данного алгоритма на языке $C++$;
4) проведено тестирование реализации на различных режимах работы программы.

Программа анализирует $WebAssembly$-файлы, корректно находит и парсит секцию кода, декодирует инструкции в соответствии с выбранным режимом (12 — контрольные инструкции, 13 — табличные и числовые, 14 — инструкции памяти, all — все группы) и формирует структурированный вывод с учётом вложенности блоков. Реализация подтвердила свою работоспособность на тестовых примерах и может быть использована для анализа и отладки WASM-модулей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лаздин А. В. Формальные языки, грамматики, автоматы. — СПБ.: Университет ИТМО, 2019. — 99 с.

2. Белоусов А. И., Ткачев С. Б. Основы современных алгоритмов. — 3-е изд. — М.: Издательство МГТУ им. Н. Э. Баумана, 2004. — 744 с.

3. WebAssembly Specification [Электронный ресурс]. — 2025. — Режим доступа: https://webassembly.github.io/spec/core/index.html (дата обращение 2025/12/20).

4. WebAssembly Specification. Values [Электронный ресурс]. — 2025. — Режим доступа: https://webassembly.github.io/spec/core/binary/values.html (дата обращение 2025/12/20).

ПРИЛОЖЕНИЕ А

```
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <string>
#include <vector>

using namespace std;

static bool fn_read_file(const char* path, vector<uint8_t>& out)
  {
  FILE* f = fopen(path, "rb");
  if (!f) return false;
  fseek(f, 0, SEEK_END);
  long sz = ftell(f);
  if (sz < 0) { fclose(f); return false; }
  fseek(f, 0, SEEK_SET);
  out.resize((size_t)sz);
  if (sz > 0) {
    if (fread(out.data(), 1, (size_t)sz, f) != (size_t)sz) {
      fclose(f);
      return false;
    }
  }
  fclose(f);
  return true;
}

static uint32_t fn_read_leb_u32(const vector<uint8_t>& b,
  size_t& off) {
  uint32_t result = 0;
  uint32_t shift = 0;
  for (;;) {
    if (off >= b.size()) { fprintf(stderr, "EOF while reading
  leb_u32\n"); exit(2); }
    uint8_t byte = b[off++];
    result |= (uint32_t)(byte & 0x7F) << shift;
    if ((byte & 0x80) == 0) break;
```

```
    shift += 7;
    if (shift >= 35) { fprintf(stderr, "LEB_u32 too large\n"); exit(2); }
  }
  return result;
}

static uint64_t fn_read_leb_u64(const vector<uint8_t>& b, size_t& off) {
  uint64_t result = 0;
  uint32_t shift = 0;
  for (;;) {
    if (off >= b.size()) { fprintf(stderr, "EOF while reading leb_u64\n"); exit(2); }
    uint8_t byte = b[off++];
    result |= (uint64_t)(byte & 0x7F) << shift;
    if ((byte & 0x80) == 0) break;
    shift += 7;
    if (shift >= 70) { fprintf(stderr, "LEB_u64 too large\n"); exit(2); }
  }
  return result;
}

static int32_t fn_read_leb_s32(const vector<uint8_t>& b, size_t& off) {
  int32_t result = 0;
  uint32_t shift = 0;
  uint8_t byte = 0;
  for (;;) {
    if (off >= b.size()) { fprintf(stderr, "EOF while reading leb_s32\n"); exit(2); }
    byte = b[off++];
    result |= (int32_t)(byte & 0x7F) << shift;
    shift += 7;
    if ((byte & 0x80) == 0) break;
    if (shift >= 35) { fprintf(stderr, "LEB_s32 too large\n"); exit(2); }
  }
  if ((shift < 32) && (byte & 0x40)) result |= (~0u << shift);
```

```
  return result;
}

static uint8_t fn_read_u8(const vector<uint8_t>& b, size_t& off) {
  if (off >= b.size()) { fprintf(stderr, "EOF while reading u8\n"); exit(2); }
  return b[off++];
}

static bool fn_verify_wasm_header(const vector<uint8_t>& b) {
  if (b.size() < 8) return false;
  if (!(b[0] == 0x00 && b[1] == 0x61 && b[2] == 0x73 && b[3] == 0x6D)) return false;
  return true;
}

static bool fn_find_section_payload(const vector<uint8_t>& b, uint8_t wanted_id, size_t& payload_off, uint32_t& payload_len) {
  if (!fn_verify_wasm_header(b)) return false;
  size_t off = 8;
  while (off < b.size()) {
    uint8_t id = fn_read_u8(b, off);
    uint32_t len = fn_read_leb_u32(b, off);
    if (off + len > b.size()) return false;
    if (id == wanted_id) {
      payload_off = off;
      payload_len = len;
      return true;
    }
    off += len;
  }
  return false;
}

static void fn_print_indent(int indent) {
  for (int i = 0; i < indent; ++i) printf("  ");
}

```

```
static const char* fn_valtype_name(uint8_t t) {
  switch (t) {
    case 0x7F: return "i32";
    case 0x7E: return "i64";
    case 0x7D: return "f32";
    case 0x7C: return "f64";
    case 0x7B: return "v128";
    case 0x70: return "funcref";
    case 0x6F: return "externref";
    case 0x40: return "empty";
    default: return "valtype?";
  }
}

static string fn_blocktype_to_string(const vector<uint8_t>& b,
   size_t& off) {
  size_t save = off;
  uint8_t peek = fn_read_u8(b, off);
  if (peek == 0x40) return "empty";
  if (peek == 0x7F || peek == 0x7E || peek == 0x7D || peek ==
   0x7C || peek == 0x7B || peek == 0x70 || peek == 0x6F)
  return string(fn_valtype_name(peek));
  off = save;
  int32_t s = fn_read_leb_s32(b, off);
  return "typeidx(" + to_string(s) + ")";
}

struct fn_ModeSet {
  bool control = false;
  bool table   = false;
  bool numeric = false;
  bool memory  = false;
};

static fn_ModeSet fn_parse_mode(int argc, char** argv) {
  fn_ModeSet m{};
  string mode = "all";
  for (int i = 1; i < argc; ++i) {
    if (strcmp(argv[i], "--mode") == 0 && i + 1 < argc) mode =
   argv[i + 1];
```

```
  }
  if (mode == "12") { m.control = true; }
  else if (mode == "13") { m.table = true; m.numeric = true; }
  else if (mode == "14") { m.memory = true; }
  else { m.control = m.table = m.numeric = m.memory = true; }
  return m;
}

struct fn_BlockStackEntry { uint8_t kind; };

static void fn_decode_control(const vector<uint8_t>& b, size_t& off, int& indent, vector<fn_BlockStackEntry>& st, uint8_t op) {
  switch (op) {
    case 0x00: fn_print_indent(indent); puts("unreachable"); break;
    case 0x01: fn_print_indent(indent); puts("nop"); break;

    case 0x02: {
      string bt = fn_blocktype_to_string(b, off);
      fn_print_indent(indent); printf("block %s\n", bt.c_str());
      st.push_back({1}); indent++;
    } break;

    case 0x03: {
      string bt = fn_blocktype_to_string(b, off);
      fn_print_indent(indent); printf("loop %s\n", bt.c_str());
      st.push_back({2}); indent++;
    } break;

    case 0x04: {
      string bt = fn_blocktype_to_string(b, off);
      fn_print_indent(indent); printf("if %s\n", bt.c_str());
      st.push_back({3}); indent++;
    } break;

    case 0x05: {
      indent--;
      fn_print_indent(indent); puts("else");
      indent++;
```

```
    } break;

    case 0x0B: {
      indent--;
      fn_print_indent(indent); puts("end");
      if (!st.empty()) st.pop_back();
    } break;

    case 0x0C: {
      uint32_t l = fn_read_leb_u32(b, off);
      fn_print_indent(indent); printf("br %u\n", l);
    } break;

    case 0x0D: {
      uint32_t l = fn_read_leb_u32(b, off);
      fn_print_indent(indent); printf("br_if %u\n", l);
    } break;

    case 0x0E: {
      uint32_t n = fn_read_leb_u32(b, off);
      fn_print_indent(indent); printf("br_table [");
      for (uint32_t i = 0; i < n; ++i) {
        uint32_t li = fn_read_leb_u32(b, off);
        printf("%u%s", li, (i + 1 < n ? ", " : ""));
      }
      uint32_t ln = fn_read_leb_u32(b, off);
      printf("] %u\n", ln);
    } break;

    case 0x0F: fn_print_indent(indent); puts("return"); break;

    case 0x10: {
      uint32_t x = fn_read_leb_u32(b, off);
      fn_print_indent(indent); printf("call %u\n", x);
    } break;

    case 0x11: {
      uint32_t y = fn_read_leb_u32(b, off);
      uint32_t x = fn_read_leb_u32(b, off);
      fn_print_indent(indent); printf("call_indirect table=%u
```

```
type=%u\n", x, y);
  } break;

  case 0x12: {
    uint32_t x = fn_read_leb_u32(b, off);
    fn_print_indent(indent); printf("return_call %u\n", x);
  } break;

  case 0x13: {
    uint32_t y = fn_read_leb_u32(b, off);
    uint32_t x = fn_read_leb_u32(b, off);
    fn_print_indent(indent); printf("return_call_indirect
table=%u type=%u\n", x, y);
  } break;

  case 0x14: {
    uint32_t x = fn_read_leb_u32(b, off);
    fn_print_indent(indent); printf("call_ref %u\n", x);
  } break;

  case 0x15: {
    uint32_t x = fn_read_leb_u32(b, off);
    fn_print_indent(indent); printf("return_call_ref %u\n", x);
  } break;

  case 0x08: {
    uint32_t x = fn_read_leb_u32(b, off);
    fn_print_indent(indent); printf("throw %u\n", x);
  } break;

  case 0x0A: fn_print_indent(indent); puts("throw_ref"); break;

  case 0x1F: {
    string bt = fn_blocktype_to_string(b, off);
    uint32_t cN = fn_read_leb_u32(b, off);
    fn_print_indent(indent); printf("try_table %s
catches=%u\n", bt.c_str(), cN);

    for (uint32_t i = 0; i < cN; ++i) {
      uint8_t ct = fn_read_u8(b, off);
```

```
        fn_print_indent(indent);
        if (ct == 0x00) {
          uint32_t tag = fn_read_leb_u32(b, off);
          uint32_t lab = fn_read_leb_u32(b, off);
          printf("  catch tag=%u label=%u\n", tag, lab);
        } else if (ct == 0x01) {
          uint32_t tag = fn_read_leb_u32(b, off);
          uint32_t lab = fn_read_leb_u32(b, off);
          printf("  catch_ref tag=%u label=%u\n", tag, lab);
        } else if (ct == 0x02) {
          uint32_t lab = fn_read_leb_u32(b, off);
          printf("  catch_all label=%u\n", lab);
        } else if (ct == 0x03) {
          uint32_t lab = fn_read_leb_u32(b, off);
          printf("  catch_all_ref label=%u\n", lab);
        } else {
          printf("  catch?(0x%02X)\n", ct);
          fprintf(stderr, "Unsupported catch tag 0x%02X\n", ct);
          exit(3);
        }
      }
      st.push_back({4}); indent++;
    } break;

    case 0xD5: {
      uint32_t l = fn_read_leb_u32(b, off);
      fn_print_indent(indent); printf("br_on_null %u\n", l);
    } break;

    case 0xD6: {
      uint32_t l = fn_read_leb_u32(b, off);
      fn_print_indent(indent); printf("br_on_non_null %u\n", l);
    } break;

    case 0xFB: {
      uint32_t sub = fn_read_leb_u32(b, off);
      if (sub == 24) {
        uint8_t castop = fn_read_u8(b, off);
        uint32_t l = fn_read_leb_u32(b, off);
        int32_t ht1 = fn_read_leb_s32(b, off);
```

```
295         int32_t ht2 = fn_read_leb_s32(b, off);
296         fn_print_indent(indent);
297         printf("br_on_cast castop=0x%02X label=%u ht1=%d
    ht2=%d\n", castop, l, ht1, ht2);
298       } else if (sub == 25) {
299         uint8_t castop = fn_read_u8(b, off);
300         uint32_t l = fn_read_leb_u32(b, off);
301         int32_t ht1 = fn_read_leb_s32(b, off);
302         int32_t ht2 = fn_read_leb_s32(b, off);
303         fn_print_indent(indent);
304         printf("br_on_cast_fail castop=0x%02X label=%u ht1=%d
    ht2=%d\n", castop, l, ht1, ht2);
305       } else {
306         fprintf(stderr, "Unsupported 0xFB subopcode %u in
    control subset\n", sub);
307         exit(3);
308       }
309     } break;
310
311     default:
312     fprintf(stderr, "Unsupported control opcode 0x%02X\n", op);
313     exit(3);
314   }
315 }
316
317 static void fn_decode_table(const vector<uint8_t>& b, size_t&
    off, int indent, uint8_t op) {
318   switch (op) {
319     case 0x25: {
320       uint32_t x = fn_read_leb_u32(b, off);
321       fn_print_indent(indent); printf("table.get %u\n", x);
322     } break;
323     case 0x26: {
324       uint32_t x = fn_read_leb_u32(b, off);
325       fn_print_indent(indent); printf("table.set %u\n", x);
326     } break;
327     default:
328     fprintf(stderr, "Unsupported table opcode 0x%02X\n", op);
329     exit(3);
330   }
```

```
}

static float fn_read_f32_ieee(const vector<uint8_t>& b, size_t& off) {
  if (off + 4 > b.size()) { fprintf(stderr, "EOF while reading f32\n"); exit(2); }
  uint32_t u = (uint32_t)b[off] | ((uint32_t)b[off+1] << 8) | ((uint32_t)b[off+2] << 16) | ((uint32_t)b[off+3] << 24);
  off += 4;
  float f;
  memcpy(&f, &u, sizeof(f));
  return f;
}

static double fn_read_f64_ieee(const vector<uint8_t>& b, size_t& off) {
  if (off + 8 > b.size()) { fprintf(stderr, "EOF while reading f64\n"); exit(2); }
  uint64_t u = 0;
  for (int i = 0; i < 8; ++i) u |= (uint64_t)b[off + i] << (8 * i);
  off += 8;
  double d;
  memcpy(&d, &u, sizeof(d));
  return d;
}

static void fn_decode_numeric_part12(const vector<uint8_t>& b, size_t& off, int indent, uint8_t op) {
  switch (op) {
    case 0x41: {
      uint32_t n = fn_read_leb_u32(b, off);
      fn_print_indent(indent); printf("i32.const %u\n", n);
    } break;
    case 0x42: {
      uint64_t n = fn_read_leb_u64(b, off);
      fn_print_indent(indent); printf("i64.const %llu\n", (unsigned long long)n);
    } break;
    case 0x43: {
```

```
      float f = fn_read_f32_ieee(b, off);
      fn_print_indent(indent); printf("f32.const %.9g\n", (double)f);
    } break;
    case 0x44: {
      double d = fn_read_f64_ieee(b, off);
      fn_print_indent(indent); printf("f64.const %.17g\n", d);
    } break;

    case 0x45: fn_print_indent(indent); puts("i32.eqz"); break;
    case 0x46: fn_print_indent(indent); puts("i32.eq"); break;
    case 0x47: fn_print_indent(indent); puts("i32.ne"); break;
    case 0x48: fn_print_indent(indent); puts("i32.lt_s"); break;
    case 0x49: fn_print_indent(indent); puts("i32.lt_u"); break;
    case 0x4A: fn_print_indent(indent); puts("i32.gt_s"); break;
    case 0x4B: fn_print_indent(indent); puts("i32.gt_u"); break;
    case 0x4C: fn_print_indent(indent); puts("i32.le_s"); break;
    case 0x4D: fn_print_indent(indent); puts("i32.le_u"); break;
    case 0x4E: fn_print_indent(indent); puts("i32.ge_s"); break;
    case 0x4F: fn_print_indent(indent); puts("i32.ge_u"); break;

    case 0x50: fn_print_indent(indent); puts("i64.eqz"); break;
    case 0x51: fn_print_indent(indent); puts("i64.eq"); break;
    case 0x52: fn_print_indent(indent); puts("i64.ne"); break;
    case 0x53: fn_print_indent(indent); puts("i64.lt_s"); break;
    case 0x54: fn_print_indent(indent); puts("i64.lt_u"); break;
    case 0x55: fn_print_indent(indent); puts("i64.gt_s"); break;
    case 0x56: fn_print_indent(indent); puts("i64.gt_u"); break;
    case 0x57: fn_print_indent(indent); puts("i64.le_s"); break;
    case 0x58: fn_print_indent(indent); puts("i64.le_u"); break;
    case 0x59: fn_print_indent(indent); puts("i64.ge_s"); break;
    case 0x5A: fn_print_indent(indent); puts("i64.ge_u"); break;

    case 0x5B: fn_print_indent(indent); puts("f32.eq"); break;
    case 0x5C: fn_print_indent(indent); puts("f32.ne"); break;
    case 0x5D: fn_print_indent(indent); puts("f32.lt"); break;
    case 0x5E: fn_print_indent(indent); puts("f32.gt"); break;
    case 0x5F: fn_print_indent(indent); puts("f32.le"); break;
    case 0x60: fn_print_indent(indent); puts("f32.ge"); break;

```

```
    case 0x61: fn_print_indent(indent); puts("f64.eq"); break;
    case 0x62: fn_print_indent(indent); puts("f64.ne"); break;
    case 0x63: fn_print_indent(indent); puts("f64.lt"); break;
    case 0x64: fn_print_indent(indent); puts("f64.gt"); break;
    case 0x65: fn_print_indent(indent); puts("f64.le"); break;
    case 0x66: fn_print_indent(indent); puts("f64.ge"); break;

    default:
    fprintf(stderr, "Unsupported numeric opcode 0x%02X (part 1-2)\n", op);
    exit(3);
  }
}

static void fn_decode_memory(const vector<uint8_t>& b, size_t& off, int indent, uint8_t op) {
  auto fn_read_memarg = [&](uint32_t& align, uint32_t& offset) {
    align = fn_read_leb_u32(b, off);
    offset = fn_read_leb_u32(b, off);
  };

  uint32_t align = 0, ofs = 0;
  switch (op) {
    case 0x28: fn_read_memarg(align, ofs); fn_print_indent(indent); printf("i32.load align=%u offset=%u\n", align, ofs); break;
    case 0x29: fn_read_memarg(align, ofs); fn_print_indent(indent); printf("i64.load align=%u offset=%u\n", align, ofs); break;
    case 0x2A: fn_read_memarg(align, ofs); fn_print_indent(indent); printf("f32.load align=%u offset=%u\n", align, ofs); break;
    case 0x2B: fn_read_memarg(align, ofs); fn_print_indent(indent); printf("f64.load align=%u offset=%u\n", align, ofs); break;
    case 0x2C: fn_read_memarg(align, ofs); fn_print_indent(indent); printf("i32.load8_s align=%u offset=%u\n", align, ofs); break;
    case 0x2D: fn_read_memarg(align, ofs); fn_print_indent(indent); printf("i32.load8_u align=%u
```

```
offset=%u\n", align, ofs); break;
429  case 0x2E: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i32.load16_s align=%u
offset=%u\n", align, ofs); break;
430  case 0x2F: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i32.load16_u align=%u
offset=%u\n", align, ofs); break;
431  case 0x30: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i64.load8_s align=%u
offset=%u\n", align, ofs); break;
432  case 0x31: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i64.load8_u align=%u
offset=%u\n", align, ofs); break;
433  case 0x32: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i64.load16_s align=%u
offset=%u\n", align, ofs); break;
434  case 0x33: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i64.load16_u align=%u
offset=%u\n", align, ofs); break;
435  case 0x34: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i64.load32_s align=%u
offset=%u\n", align, ofs); break;
436  case 0x35: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i64.load32_u align=%u
offset=%u\n", align, ofs); break;
437
438  case 0x36: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i32.store align=%u
offset=%u\n", align, ofs); break;
439  case 0x37: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i64.store align=%u
offset=%u\n", align, ofs); break;
440  case 0x38: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("f32.store align=%u
offset=%u\n", align, ofs); break;
441  case 0x39: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("f64.store align=%u
offset=%u\n", align, ofs); break;
442  case 0x3A: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i32.store8 align=%u
```

```
offset=%u\n", align, ofs); break;
443     case 0x3B: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i32.store16 align=%u
offset=%u\n", align, ofs); break;
444     case 0x3C: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i64.store8 align=%u
offset=%u\n", align, ofs); break;
445     case 0x3D: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i64.store16 align=%u
offset=%u\n", align, ofs); break;
446     case 0x3E: fn_read_memarg(align, ofs);
fn_print_indent(indent); printf("i64.store32 align=%u
offset=%u\n", align, ofs); break;
447
448     case 0x3F: {
449       uint8_t memidx = fn_read_u8(b, off);
450       fn_print_indent(indent); printf("memory.size %u\n",
(unsigned)memidx);
451     } break;
452
453     case 0x40: {
454       uint8_t memidx = fn_read_u8(b, off);
455       fn_print_indent(indent); printf("memory.grow %u\n",
(unsigned)memidx);
456     } break;
457
458     default:
459     fprintf(stderr, "Unsupported memory opcode 0x%02X\n", op);
460     exit(3);
461   }
462 }
463
464 static void fn_parse_code_section_10(const vector<uint8_t>& b,
size_t payload_off, uint32_t payload_len, const fn_ModeSet&
mode) {
465   size_t off = payload_off;
466   size_t end = payload_off + payload_len;
467
468   uint32_t func_count = fn_read_leb_u32(b, off);
469   printf("CodeSection(id=10): functions=%u\n", func_count);
```

```
470
471  for (uint32_t fi = 0; fi < func_count; ++fi) {
472    uint32_t body_size = fn_read_leb_u32(b, off);
473    size_t body_start = off;
474    size_t body_end = off + body_size;
475    if (body_end > end) { fprintf(stderr, "Function body
     overruns section\n"); exit(2); }
476
477    printf("\nfunc[%u]: body_size=%u\n", fi, body_size);
478
479    uint32_t local_decl_cnt = fn_read_leb_u32(b, off);
480    printf("  locals_decls=%u\n", local_decl_cnt);
481    for (uint32_t i = 0; i < local_decl_cnt; ++i) {
482      uint32_t n = fn_read_leb_u32(b, off);
483      uint8_t t = fn_read_u8(b, off);
484      printf("    local_decl: count=%u type=%s(0x%02X)\n", n,
     fn_valtype_name(t), t);
485    }
486    int indent = 1;
487    vector<fn_BlockStackEntry> st;
488    st.push_back({255});
489
490    while (off < body_end) {
491      uint8_t op = fn_read_u8(b, off);
492      if (mode.control) {
493        if (op == 0x00 || op == 0x01 || op == 0x02 || op == 0x03
     || op == 0x04 || op == 0x05 ||
494        op == 0x08 || op == 0x0A || op == 0x0B || op == 0x0C ||
     op == 0x0D || op == 0x0E ||
495        op == 0x0F || op == 0x10 || op == 0x11 || op == 0x12 ||
     op == 0x13 || op == 0x14 ||
496        op == 0x15 || op == 0x1F || op == 0xD5 || op == 0xD6 ||
     op == 0xFB) {
497          fn_decode_control(b, off, indent, st, op);
498          if (op == 0x0B && st.empty()) break;
499          if (op == 0x0B && st.size() == 0) break;
500          if (op == 0x0B && st.empty()) break;
501          if (op == 0x0B && off == body_end) break;
502          continue;
503        }
```

```
      }

      if (mode.table && (op == 0x25 || op == 0x26)) {
        fn_decode_table(b, off, indent, op);
        continue;
      }

      if (mode.memory &&
      ((op >= 0x28 && op <= 0x3E) || op == 0x3F || op == 0x40)) {
        fn_decode_memory(b, off, indent, op);
        continue;
      }

      if (mode.numeric &&
      ((op >= 0x41 && op <= 0x44) || (op >= 0x45 && op <= 0x66))) {
        fn_decode_numeric_part12(b, off, indent, op);
        continue;
      }

      fprintf(stderr, "Unsupported opcode 0x%02X at func[%u] offset=%zu (within body)\n", op, fi, (size_t)(off - body_start));
      exit(3);
    }

    if (off != body_end) {
      fprintf(stderr, "Warning: func[%u] decoded %zu bytes, expected %u (body_end mismatch)\n",
      fi, (size_t)(off - body_start), body_size);
      off = body_end;
    }
  }

  if (off != end) {
    fprintf(stderr, "Warning: section payload not fully consumed (%zu bytes left)\n", (size_t)(end - off));
  }
}
```

```
539 static void fn_usage() {
540   puts("Usage: wasm_parser <file.wasm> [--mode 12|13|14|all]");
541 }
542
543 int main(int argc, char** argv) {
544   if (argc < 2) { fn_usage(); return 1; }
545   const char* path = argv[1];
546
547   fn_ModeSet mode = fn_parse_mode(argc, argv);
548
549   vector<uint8_t> bytes;
550   if (!fn_read_file(path, bytes)) {
551     fprintf(stderr, "Cannot read file: %s\n", path);
552     return 1;
553   }
554   if (!fn_verify_wasm_header(bytes)) {
555     fprintf(stderr, "Not a valid wasm (bad header)\n");
556     return 1;
557   }
558
559   size_t payload_off = 0;
560   uint32_t payload_len = 0;
561   if (!fn_find_section_payload(bytes, 10 /*code*/, payload_off,
       payload_len)) {
562     fprintf(stderr, "Section id=10 (code) not found\n");
563     return 1;
564   }
565
566   fn_parse_code_section_10(bytes, payload_off, payload_len,
       mode);
567   return 0;
568 }
569
570
```

Листинг A.1 — Программная реализация